

Deep Learning

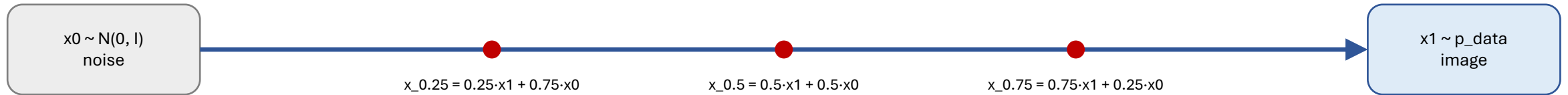
Lecture 10b — Flow matching, modern image and video generators
(2026)

Recap and plan

- Lecture 10 covered DDPM, score functions, classifier(-free) guidance and latent diffusion — not repeated here
- Today: the same goal with a **straight-line path** and a plain **regression** loss (flow matching); why it is a valid generative model; what 2025–26 image and video generators look like; sampling in one step; world models; diffusion language models
- Own slides plus a few pages adapted (editable) from Stanford CS231n Lecture 14 (May 2026) and MIT 6.S184 (2026); all sources and arXiv ids are in the notes

Flow matching in one picture

velocity along the line is constant: $v = x_1 - x_0 \rightarrow$ regress $v_\theta(x_t, t)$ onto $x_1 - x_0$



sample: $x \leftarrow x + \Delta t \cdot v_\theta(x, t)$, from $t = 0$ to 1

Draw a noise sample x_0 and a data sample x_1 ; interpolate $x_t = t \cdot x_1 + (1-t) \cdot x_0$

The velocity along that line is constant: $x_1 - x_0$ — a regression target with no noise schedule, no ELBO

Train: $L(\theta) = E \|v_\theta(x_t, t) - (x_1 - x_0)\|^2$; sample: Euler steps of the ODE $dx/dt = v_\theta(x, t)$ from $t = 0$ to 1

Time conventions:

“Flow time convention”:

- Data: $t = 1$
- Noise: $t = 0$

“Diffusion time convention”:

- Data $t = 0$
- Noise: $t \rightarrow \text{infinity}$

“Discrete time”:

- Use discrete time steps instead of continuous time steps
- No ODE or SDE but Markov chain
- DDIM $\sim$ Probability flow ODE

Flow matching, rectified flows, stochastic interpolants

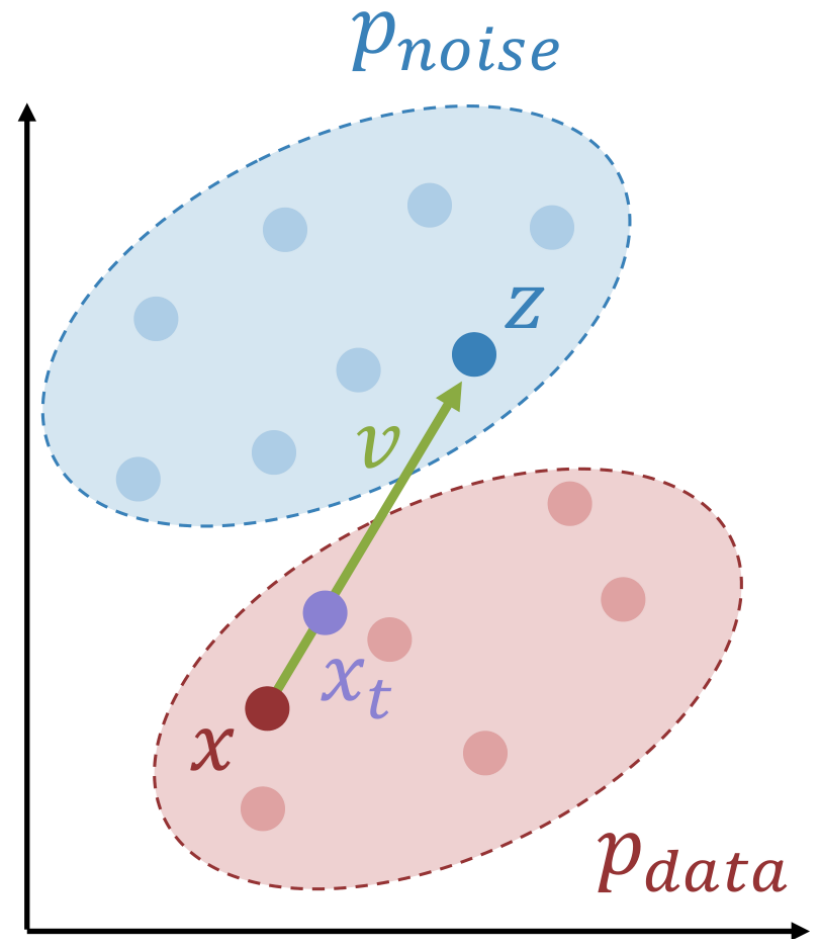
Score-based diffusion models with SDEs

DDPM
DDIM

Diffusion Models: Rectified Flow

Core training loop is just
a few lines of code!

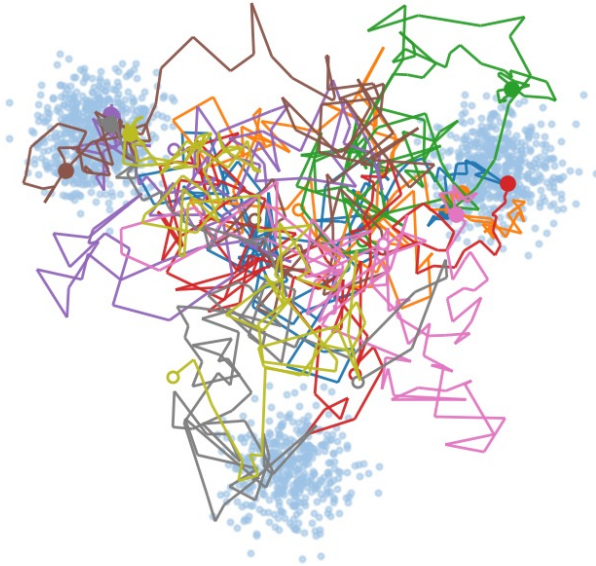
```
for x in dataset:
    z = torch.randn_like(x)
    t = random.uniform(0, 1)
    xt = (1 - t) * x + t * z
    v = model(xt, t)
    loss = (z - x - v).square().sum()
```



Liu et al, "Flow Straight and Fast: Learning to Generate and Transfer Data with Rectified Flow", 2022
Lipman et al, "Flow Matching for Generative Modeling", 2022

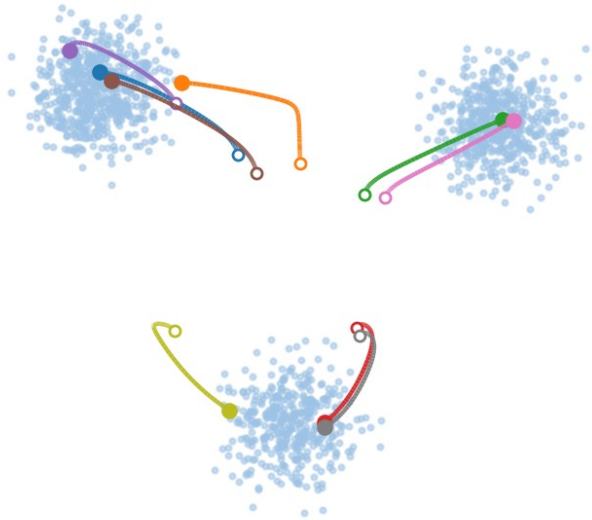
Why straight paths? Three samplers on the same toy data

Diffusion, stochastic sampling (DDPM)
1000 steps, paths wander



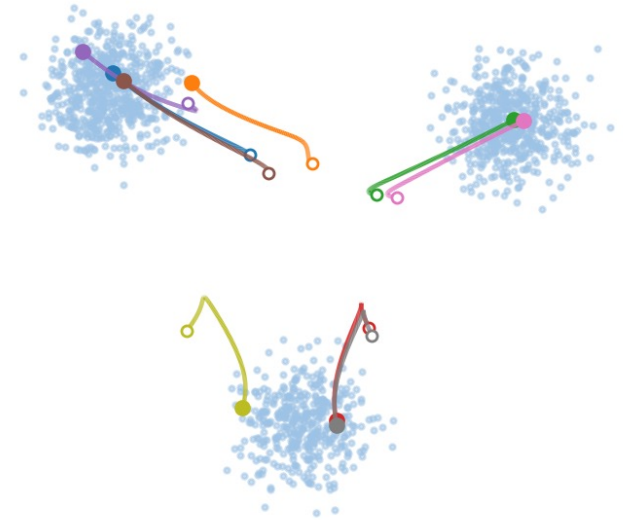
○ noise sample ● generated sample

Diffusion, probability-flow ODE (DDIM)
deterministic but curved



○ noise sample ● generated sample

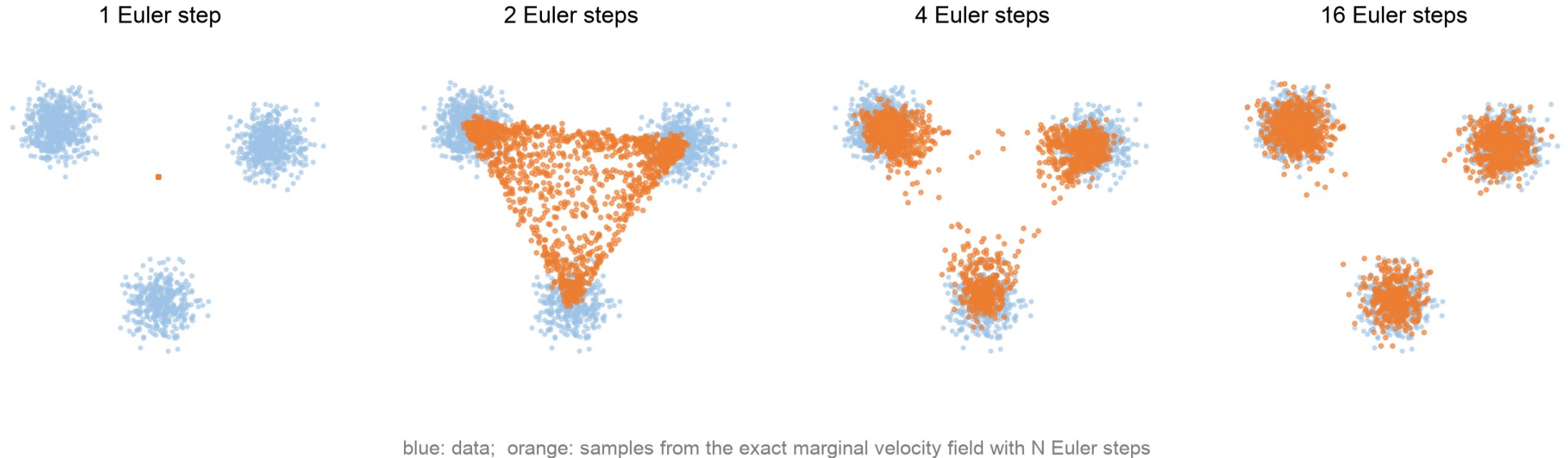
Flow matching, linear path
nearly straight



○ noise sample ● generated sample

Left: DDPM ancestral sampling — 1000 stochastic steps, paths wander before landing
Middle: the probability-flow ODE of the same diffusion (DDIM) — deterministic but curved
Right: flow matching with the linear path — nearly straight, so a few Euler steps suffice

Few steps work only if the marginal field is nearly straight



Even with the *exact* marginal velocity, one Euler step lands on the conditional mean, not on the data
Where conditional paths cross, the marginal field curves — this is what reflow, distillation and one-step models fix

Optimal Prediction

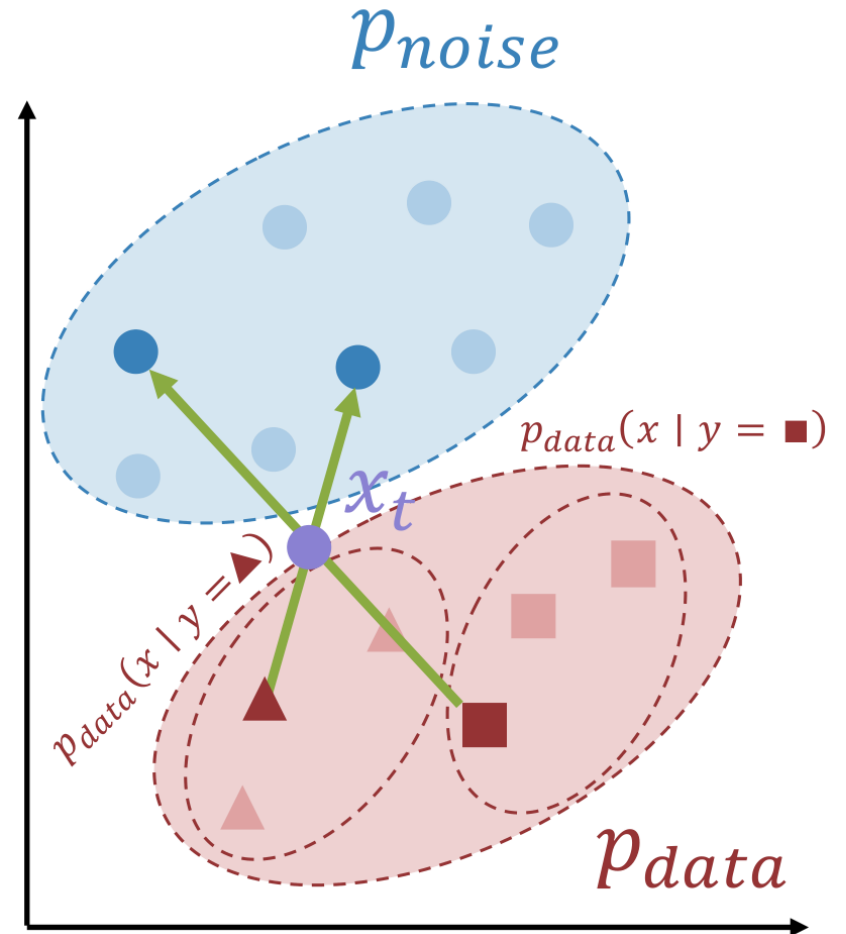
Training

```
for (x, y) in dataset:
    z = torch.randn_like(x)
    t = random.uniform(0, 1)
    xt = (1 - t) * x + t * z
    if random.random() < 0.5: y = y_null
    v = model(xt, y, t)
    loss = (z - x - v).square().sum()
```

Q: What is the optimal prediction for the network?

There may be many pairs (x, z) that give the same x_t ; network must average over them

Full noise ($t=1$) is easy: optimal v is mean of p_{data} No noise ($t=0$) is easy: optimal v is mean of p_{noise} Middle noise is hardest, most ambiguous But we give equal weight to all noise levels! Solution: Use a non-uniform noise schedule



Dhariwal and Nichol, "Diffusion Models beat GANs on Image Synthesis", arXiv 2021
Ho and Salimans, "Classifier-Free Diffusion Guidance", arXiv 2022

Why does regressing on conditional velocities give a generative model?

- We only ever see pairs (x_0, x_1) : each pair defines a **conditional** straight path and velocity
- The network must output one velocity per (x, t) : it learns the **marginal** velocity — the average of all conditional velocities passing through x at time t
- Theorem (Lipman et al., 2023): the marginal velocity transports the noise distribution exactly onto the data distribution, and the conditional regression loss has the same gradient as the marginal one

The Flow Matching Matrix

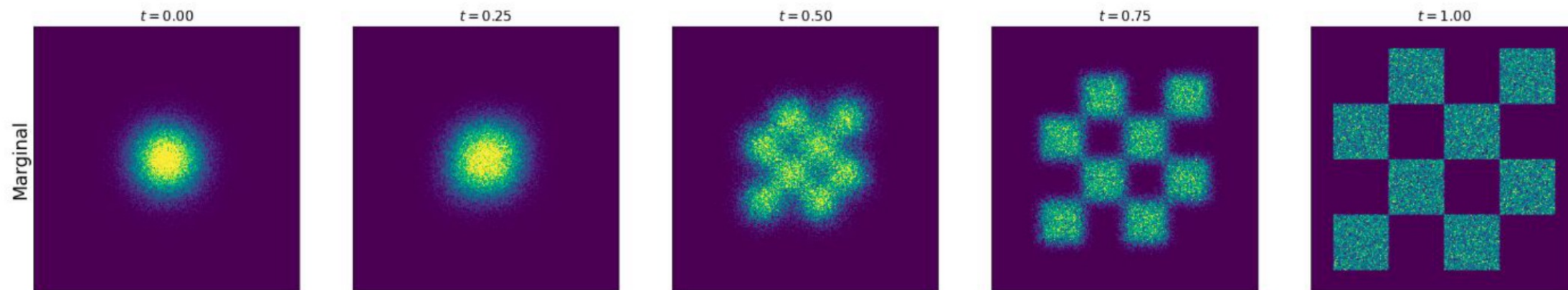
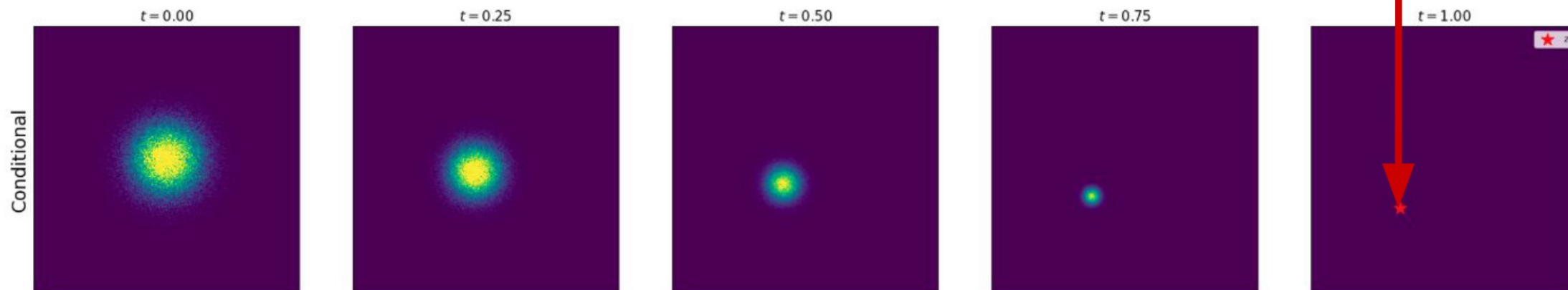


“Conditional” = “Per single data point”

“Marginal” = “Across distribution of data points”

p_{init}

Conditional Probability Path $p_t(\cdot|z)$

 z  p_{init}

Marginal Probability Path p_t

 p_{data}

Marginal Prob. Path, Vector Field, and Score

	Notation	Key property	Formula
Marginal Probability Path	p_t	Interpolates p_{init} and p_{data}	$\int p_t(x z)p_{\text{data}}(z)dz$
Marginal Vector Field	$u_t^{\text{target}}(x)$	ODE follows marginal path	$\int u_t^{\text{target}}(x z) \frac{p_t(x z)p_{\text{data}}(z)}{p_t(x)} dz$

;

Algorithm 4 Flow Matching Training for CondOT path

Require: A dataset of samples $z \sim p_{\text{data}}$, neural network u_t^θ

- 1: **for** each mini-batch of data **do**
- 2: Sample a data example z from the dataset.
- 3: Sample a random time $t \sim \text{Unif}_{[0,1]}$.
- 4: Sample noise $\epsilon \sim \mathcal{N}(0, I_d)$
- 5: Set $x = tz + (1 - t)\epsilon$
- 6: Compute loss

$$\mathcal{L}(\theta) = \|u_t^\theta(x) - (z - \epsilon)\|^2$$

- 7: Update the model parameters θ via gradient descent on $\mathcal{L}(\theta)$.
 - 8: **end for**
-

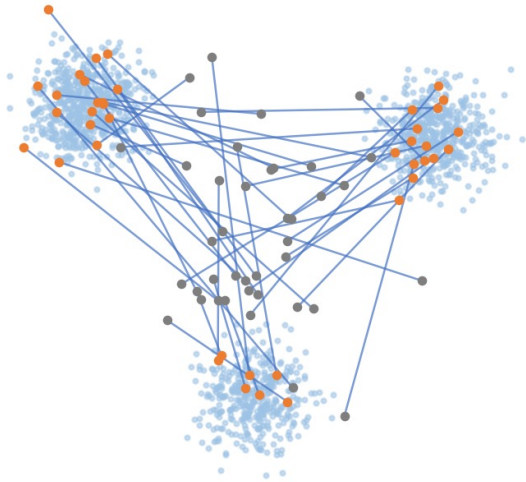
Marginal Prob. Path, Vector Field, and FM Loss

	Notation	Key property	Formula
Marginal Probability Path	p_t	Interpolates p_{init} and p_{data}	$\int p_t(x z)p_{\text{data}}(z)dz$
Marginal Vector Field	$u_t^{\text{target}}(x)$	ODE follows marginal path	$\int u_t^{\text{target}}(x z) \frac{p_t(x z)p_{\text{data}}(z)}{p_t(x)} dz$
Marginal FM Loss	$L_{\text{FM}}(\theta)$	Implicitly minimized via cond FM loss	$\mathbb{E}_{t,z,x} [\ u_t^\theta(x) - u_t^{\text{target}}(x)\ ^2]$

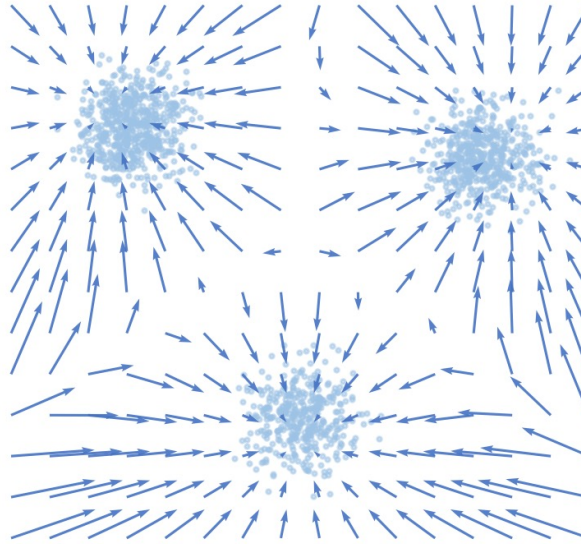
None of these objects are tractable. But we can still learn them!

Conditional targets vs. the marginal field (toy data)

Conditional paths: one straight line per (x_0, x_1) pair
(independent coupling: lines cross)



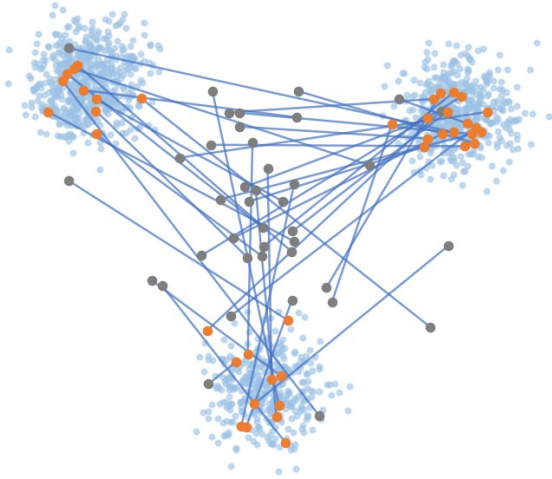
Marginal velocity $u_t(x) = E[x_1 - x_0 \mid x_t = x]$ ($t = 0.5$)
— the average the network must learn (x_0 : noise, x_1 : data)



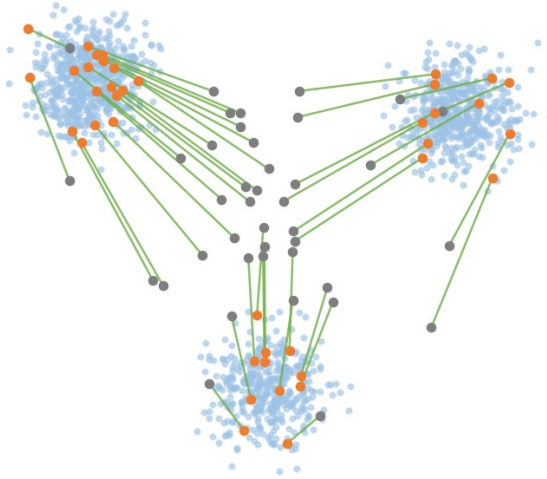
- Left: each training pair contributes one straight line; with independent pairing the lines cross
- Right: the marginal velocity $u_t(x) = E[x_1 - x_0 \mid x_t = x]$ the network actually learns ($t = 0.5$, computed exactly)

Rectified flow and reflow

Independent coupling (x_0, x_1): crossing lines
→ marginal flow must curve



Reflow coupling ($x_0, \text{Flow}(x_0)$): no crossings
→ retrain on these pairs: straight, 1–2 steps



- Three groups, one algorithm (ICLR 2023): flow matching (Lipman), rectified flow (Liu), stochastic interpolants (Albergo)
- **Reflow**: generate pairs ($x_0, \text{Flow}(x_0)$) with the trained model and retrain on them — pairs no longer cross, the field straightens, 1–2 Euler steps become accurate
- Used by InstaFlow and as the 'rectified' in Stable Diffusion 3

Diffusion is the Gaussian-path special case

- Choose the Gaussian path $x_t = \alpha_t x_1 + \sigma_t x_0$ instead of the straight line: flow matching becomes DDPM/score matching with a different parameterization
- Velocity, score, ε -prediction and x -prediction are linear functions of each other — 'two sides of the same coin' (Gao et al., DeepMind blog, Dec 2024)
- Deterministic diffusion sampling (DDIM, probability-flow ODE) *is* a flow; what is new is the straight path, the simpler loss and the freedom to choose the timestep weighting

Reparameterization: Velocity Field $\rightarrow$ Score Function

$$a_t = \left(\beta_t^2 \frac{\dot{\alpha}_t}{\alpha_t} - \dot{\beta}_t \beta_t \right), \quad b_t = \frac{\dot{\alpha}_t}{\alpha_t}$$

$$u_t^{\text{target}}(x|z) = a_t \nabla \log p_t(x|z) + b_t x$$

$$u_t^{\text{target}}(x) = a_t \nabla \log p_t(x) + b_t x$$

Proof: Algebra. Insert formulas. See lecture notes.

Early Diffusion Models learnt the score function instead and then just transformed it into the vector field! This is equivalent!

Noise Schedules

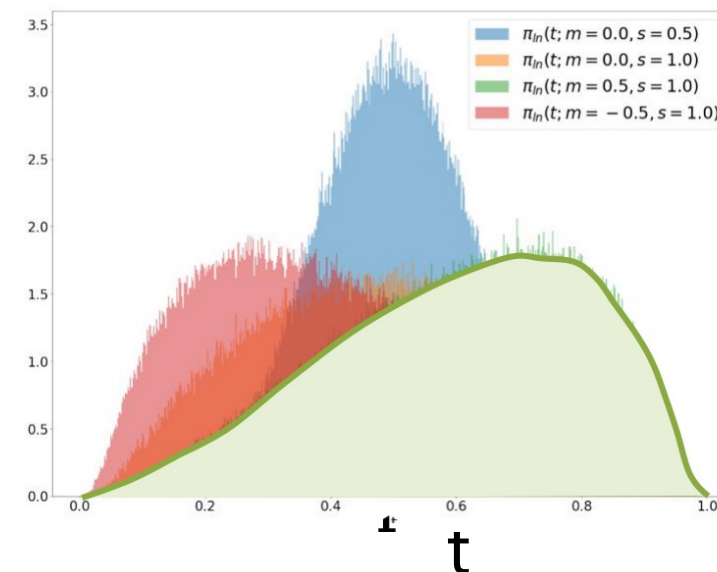
Training

```
for (x, y) in dataset:
    z = torch.randn_like(x)
    t = torch.randn(()).sigmoid()
    xt = (1 - t) * x + t * z
    if random.random() < 0.5: y = y_null
    v = model(xt, y, t)
    loss = (z - x - v).square().sum()
```

There may be many pairs (x, z) that give the same xt ;
network must average over them

Full noise ($t=1$) is easy: optimal v is mean of p_{data} No noise ($t=0$)
is easy: optimal v is mean of p_{noise} Middle noise is hardest,
most ambiguous But we give equal weight to all noise levels!
Solution: Use a non-uniform noise schedule

$p(t)$



Put more emphasis on middle noise

Common choice: logit-normal sampling

For high-res data, often shift to higher noise to account for pixel correlations

Esser et al, “Scaling Rectified Flow Transformers for High-Resolution Image Synthesis”, arXiv 2024

Guidance carries over unchanged

- Train with the condition randomly dropped; sample with $v = (1-w) \cdot v(x|\emptyset) + w \cdot v(x|y)$ — classifier-free guidance (Lecture 10, slides 59–62) as a combination of velocities
- Latent generation (Lecture 10, slide 63) is unchanged too: run the flow in the VAE latent space
- Almost every deployed image/video model uses guidance (SD3: $w \approx 4$); it trades diversity for prompt adherence

The text-to-image design space in 2026

latent space × architecture × text encoder × training paradigm

Four independent choices

Training paradigm	Latent space	Architecture	Text encoder	Text conditioning
DDPM / score (Lecture 10)	VQ-VAE / VQGAN tokens	U-Net (Lecture 10)	CLIP	cross-attention
flow matching / rectified flow	continuous VAE (4–16 ch)	DiT (patchified latents)	CLIP + T5	double-stream MM-DiT
+ preference RL (Flow-GRPO)	VAE aligned to DINO features (Flux 2)	MM-DiT (text+image streams)	an LLM / VLM (Qwen3-VL)	single-stream MM-DiT
+ distillation / flow maps	pixels again (JiT)	one transformer, AR + diffusion	the same model (native)	native multimodal

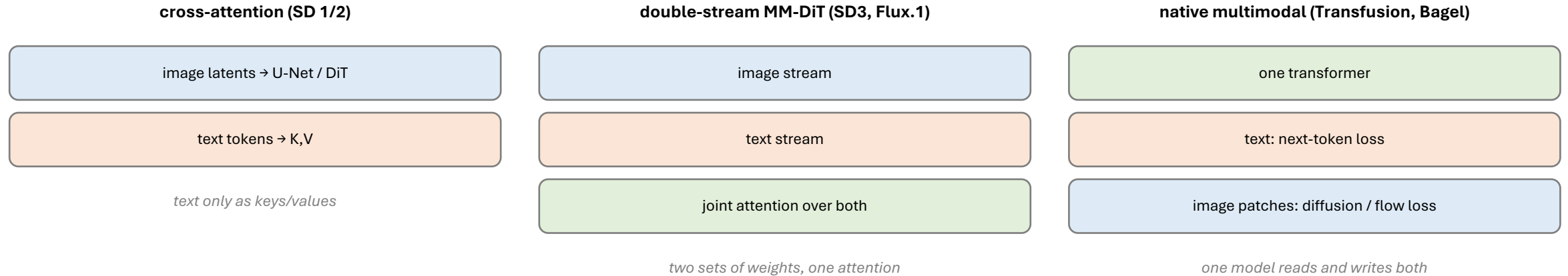
rows are not aligned recipes — each column is an independent choice; the last row is the 2025-26 frontier

Every recent model is a point in this grid; the frontier moved down each column between 2022 and 2026

Training: DDPM → flow matching → + preference RL (Flow-GRPO, later) → + distillation

Latent: VQ tokens → continuous VAE (4 → 16 channels) → VAE aligned to vision features → pixels again (JiT)

How text meets image: cross-attention → MM-DiT → native multimodal



SD 1/2: text tokens only as keys/values of cross-attention inside a U-Net

SD3 / Flux.1 (2024): a Diffusion Transformer with separate weights for text and image tokens and joint attention over both (MM-DiT); Flux.1 merges streams in later blocks

Transfusion (2024), Bagel (2025): one transformer trained with next-token loss on text and diffusion/flow loss on image patches — reads and writes both

Text encoders and latent spaces moved too

- **Text:** CLIP (2022) → CLIP + T5-XXL (SD3, 2024) → an LLM/VLM as the text encoder (Qwen-Image, Flux 2, 2025) — prompts are understood, not just embedded
- **Latents:** the VAE is a three-way trade-off (perceptual quality, compression, learnability); 2025 models add channels (16–32) and align latents to a strong vision encoder — the REPA idea applied to the VAE
- **Backbone:** U-Net → DiT with adaLN time conditioning → MM-DiT; **objective:** rectified flow with logit-normal t and a resolution-dependent shift

Model lineage 2022 → 2025

Model family	Latent	Backbone	Text	Objective
Stable Diffusion 1/2 (2022)	VAE 4-ch	U-Net + cross-attn	CLIP	DDPM (ϵ -prediction)
SD3 / Flux.1 (2024)	VAE 16-ch	MM-DiT (double stream)	CLIP + T5	rectified flow, logit-normal t
Flux 2, Qwen-Image, Z-Image (2025)	VAE aligned to vision features	MM-DiT (single/merged stream)	VLM / LLM	flow matching
Transfusion, Hunyuan 3.0, Bagel (2024-25)	VAE / patches	one transformer (AR + diffusion)	the same model	next-token + diffusion
GPT-4o images, Nano Banana (2025)	not public	chat model + image decoder (reported)	the same model	not public

Each generation swapped one or two columns; the recipe is otherwise stable

The last row is not public: OpenAI and Google describe image generation inside the chat model, presumably an autoregressive model with a flow/diffusion decoder

Case Study: Stable Diffusion 3

Image source: Scaling Rectified Flow
Transformers for High-Resolution Image
Synthesis [1]

Overview:

- Flow Matching model with “straight line” schedulers (CondOT path)
- Classifier-free guidance with weight 2.0 - 5.0
- Flow Matching in latent space (use pre-trained VAE)
- Number of parameters of model: 8 billion
- Number of simulation/sampling steps: 50
- Dataset: [LAION](#)



REPA: representation alignment makes DiT training much faster

- Diffusion transformers learn poor semantic features on their own; add a loss that aligns an intermediate DiT layer with **DINOv2** features of the clean image (Lecture 5)
- Yu et al. (ICLR 2025): SiT-XL training speeds up by **more than 17.5×** — 400k steps match a 7M-step model; FID 1.42 on ImageNet 256 with guidance interval
- Follow-ups: REPA-E (2025) trains the VAE end-to-end with the alignment loss; Representation Autoencoders (Oct 2025) diffuse directly in a frozen DINOv2/SigLIP latent; Flux 2 aligns its VAE latents the same way
- Back to basics (JiT, Li & He, Nov 2025): with x-prediction, plain pixel-space transformers work again

Sampling in few steps — and in one

solvers, distillation, consistency models, flow maps

From many steps to one: distillation, consistency, flow maps

MeanFlow / flow map: learn the average velocity between any two times (r, t)



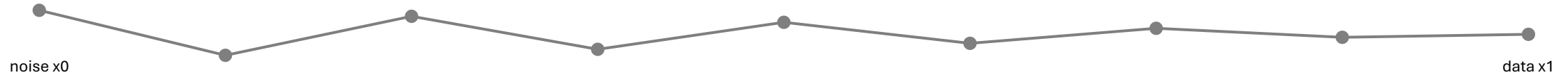
consistency model: any point on the trajectory $\rightarrow$ its end point (one step)



progressive distillation: one student step = two teacher steps; repeat



teacher: an ODE solver takes many small steps along a curved trajectory



Solvers (Heun, DPM-Solver): 10–50 steps with no retraining

Progressive distillation (Salimans & Ho, 2022): a student matches two teacher steps with one; repeat

Consistency models (Song et al., 2023): map any point on a trajectory to its end point; train by distillation or from scratch

Flow maps (2025): learn the solution operator $\Phi(x, r, t)$ directly; **MeanFlow** learns the average velocity over (r, t) from scratch

Catalog of one-step generative models

- VAEs

- Stable training (maximum likelihood)
- Tractable likelihood estimation
- Low sample quality

- GANs

- Unstable training (adversarial games)
- High sample quality
- No likelihoods

- Normalizing flows

- Stable training (maximum likelihood)
- Restricted model architecture
- Low sample quality

- Consistency models

- Stable training (pseudo-objective)
- High sample quality
- No likelihoods
- Moderate architecture constraints.

One-step generation in 2025–26

- **MeanFlow** (May 2025): average-velocity field trained from scratch; 1 network evaluation, FID 3.43 on ImageNet 256×256
- **Continuous-time consistency models** (sCM, 2024): 2-step samples at ImageNet 512 with about 1/10 of the teacher's sampling compute
- **Drifting models** (Deng, Li, Li, Du, He, Feb 2026): a one-step generator trained by a drift field; FID 1.54 (latent) / 1.61 (pixels) on ImageNet 256 — on par with multi-step models
- The number of steps is now a design choice, not a limitation of the framework

RL fine-tuning of flow models

GRPO from Lecture 12 meets deterministic samplers

Why would we want stochastic/SDE dynamics?

In theory: All diffusion coefficients lead to the same result (sample from data distribution).

In practice:

- **Training error:** Neural network has not perfectly learnt the marginal vector field/score.
- **Simulation error:** We need to simulate SDE/ODE leading to discretization error.

Downstream applications: Fine-tuning, inference-time optimization, etc. might require stochastic evolution

**Good news: ODE sampling often leads to the best results.
Therefore, SDE sampling is an option, not a must!**

Flow-GRPO and TempFlow-GRPO

ODE sampler: same noise $x_0 \rightarrow$ always the same image (no exploration for RL) SDE sampler: same noise $x_0 \rightarrow$ a distribution of images (schematic; Flow-GRPO uses an SDE with the same marginals)

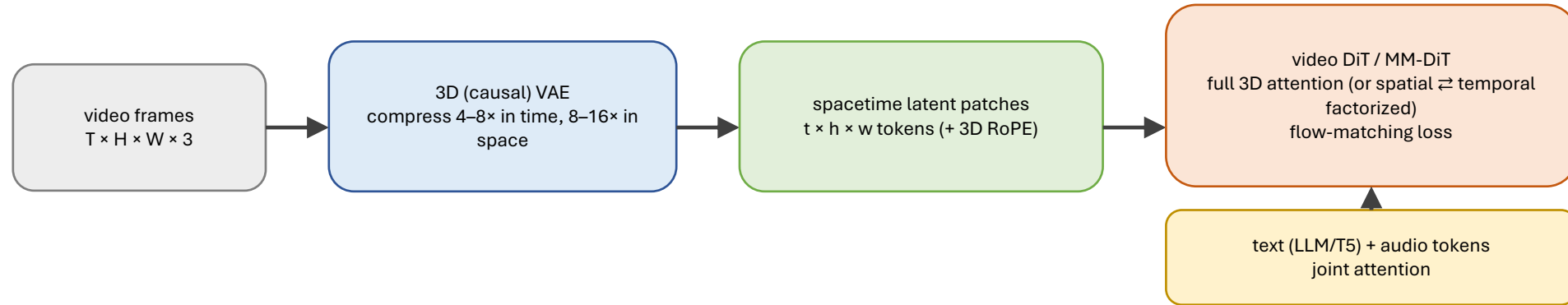


- An ODE sampler is deterministic: the same noise always gives the same image — nothing to explore
- **Flow-GRPO** (Liu et al., May 2025): convert the ODE into an SDE with the same marginals, sample several images per prompt, reward with a verifier (GenEval 63% $\rightarrow$ 95%), update with GRPO
- **TempFlow-GRPO** (He et al., Aug 2025): credit should not be spread uniformly over timesteps — branch trajectories at chosen steps and weight updates by each step's noise level

Video generation and world models

the same recipe with a time axis

Video generators are latent flow-matching transformers with a time axis



Sora (Feb 2024), Movie Gen (Oct 2024, 30B), Wan 2.x, LTX-2, Veo 3 and Sora 2 (2025) all follow this template; older: 3D U-Nets with factorized attention

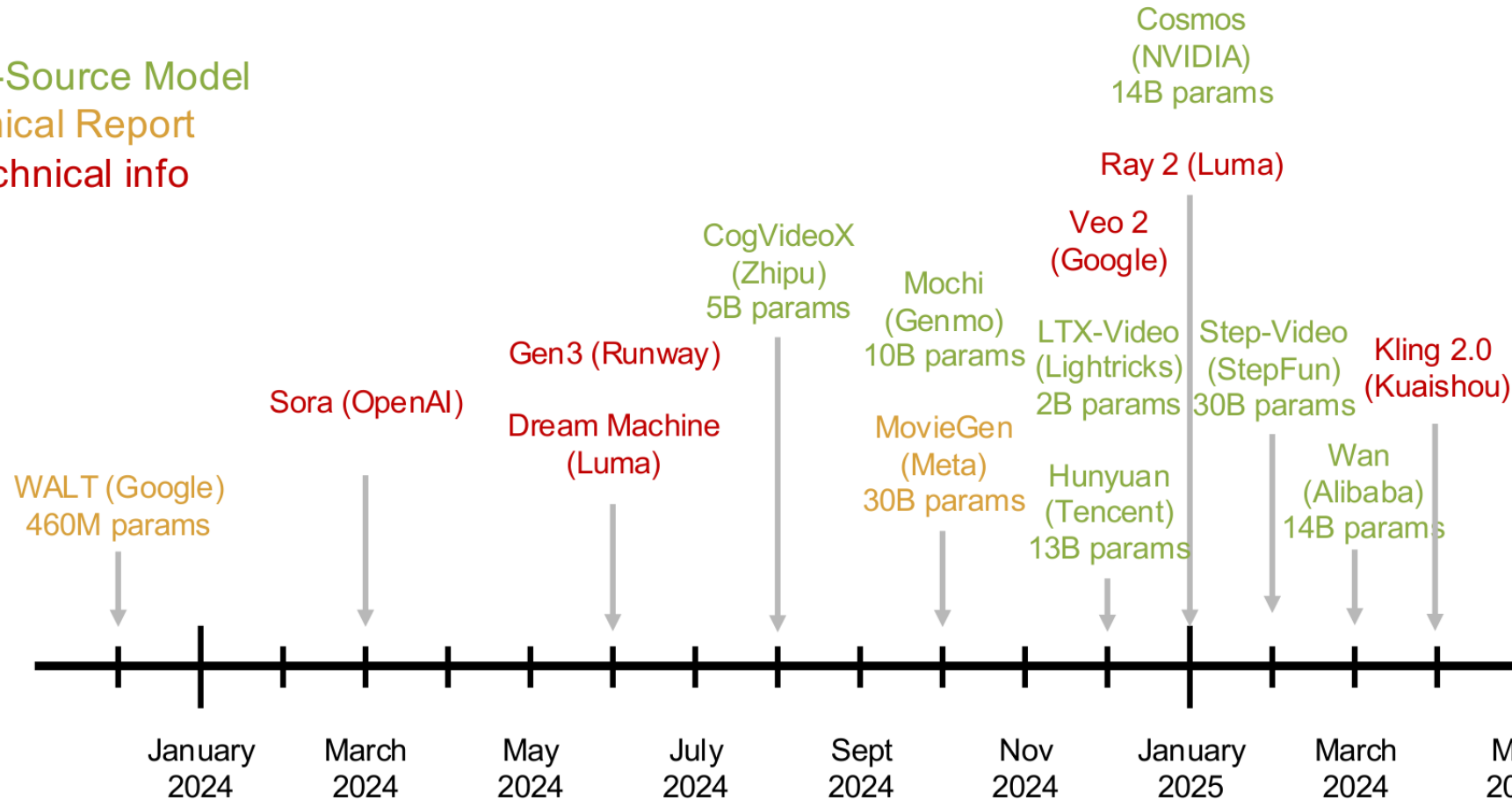
A 3D VAE compresses time and space; spacetime patches become tokens with 3D positional encoding
Full 3D attention (or spatial ⇌ temporal factorized attention in older 3D U-Nets); text and now audio tokens attend jointly with video tokens
Sampling and RL tricks carry over from images: few-step distillation, CFG, reward fine-tuning

The Era of Video Diffusion Models

Open-Source Model

Technical Report

No technical info



Gupta et al., "Photorealistic Video Generation with Diffusion Models", arXiv 2023 (Dec)
OpenAI, "Sora: Creating Video from Text", 2024 (Feb)
Polyak et al., "Movie Gen: A Cast of Media Foundation Models", arXiv 2024 (Oct)
Kong et al., "HunyuanVideo: A Systematic Framework for Large Video Generation Models", arXiv 2024 (Dec)
NVIDIA, "Cosmos World Foundation Model Platform for Physical AI", arXiv 2025 (Jan)
Team Wan, "Wan: Open and Advanced Large-Scale Video Generative Models", arXiv 2025 (March)

Video generators in 2025–26

- Closed: Veo 3 (Google, May 2025, native audio), Sora 2 (OpenAI, Sep 2025); open weights: Wan 2.x (Alibaba, MoE-DiT), LTX-2 (Lightricks, Oct 2025, joint audio-video), HunyuanVideo
- Training cost is falling: open reproductions (OpenSora) report hundreds of thousands of GPU-hours, not millions
- Video models are the substrate for world models — next slide

World models: three paradigms

Paradigm	What is generated	Examples	Use
3D scene generation	an explicit scene (Gaussian splats / NeRF) to render	Marble (World Labs, Nov 2025), DreamFusion, GSGen	assets, simulation
interactive video generation	the next frame given the user's action	Genie 1 → 3 (DeepMind, 2024–Aug 2025), GameNGen, Oasis	playable worlds, robot data
latent world models	the next latent state, not pixels	V-JEPA 2 (Meta, Jun 2025), PAN	planning, control

Genie 3: real-time 720p interactive worlds lasting minutes, from a text prompt; limitations reported: short memory, latent actions, no explicit geometry

A world model predicts what happens next given an action: generated 3D scenes, interactive video, or latent states

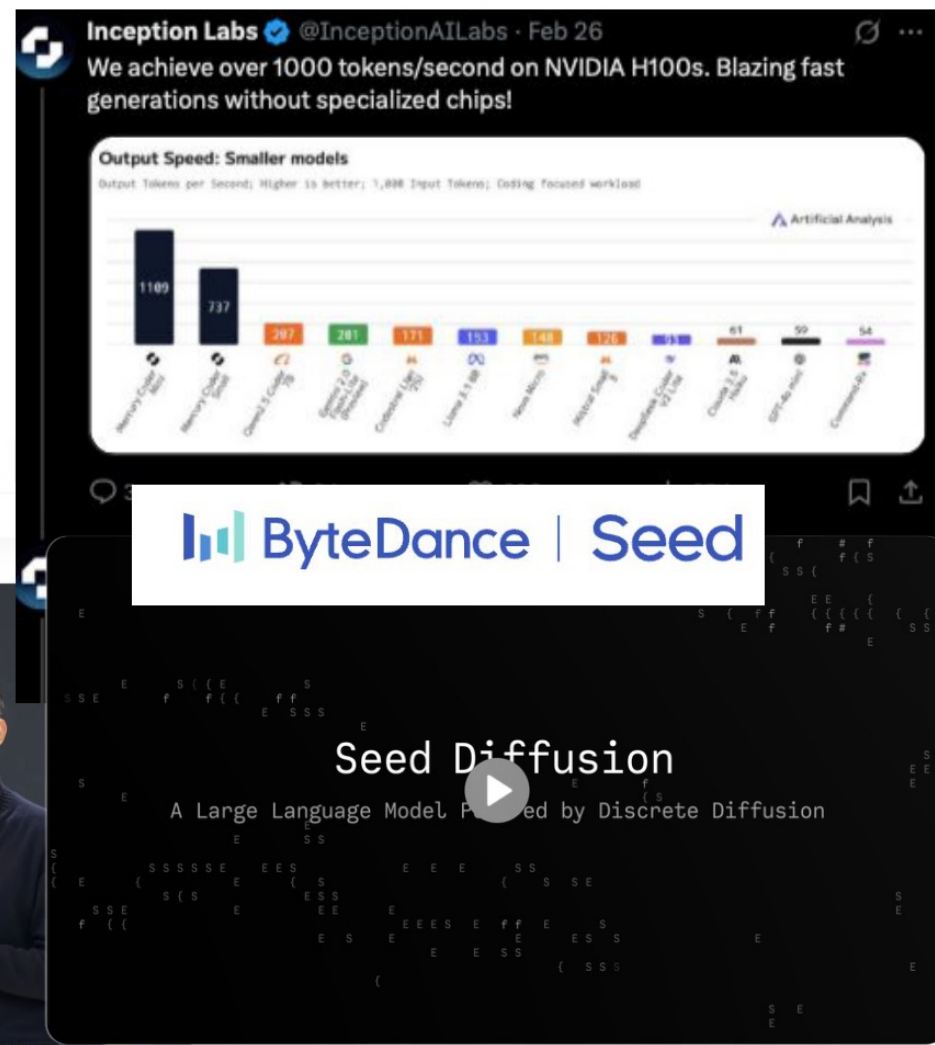
Genie (DeepMind): learns latent actions from unlabeled video; Genie 3 (Aug 2025) generates playable 720p worlds in real time

V-JEPA 2 (Meta, Jun 2025): non-generative — predicts masked latents, then plans with the predictor (ties to Lecture 5's JEPA)

Flows for discrete data: diffusion language models

and why autoregressive models struck back for images

Discrete Diffusion Models in the news



Slide: Subham Sahoo

Diffusion LLMs generate text in arbitrary order

MDLM

Sampling step: 00/30

Austin et al., “Structured Denoising Diffusion Models in Discrete State-Spaces”, NeurIPS 2022

Masked Diffusion LLMs generate text

MDLM

Sampling step: 00/30

Austin et al., “Structured Denoising Diffusion Models in Discrete State-Spaces”, NeurIPS 2022

Models

Discussion: Discrete Diffusion Models vs Autoregressive

Advantages

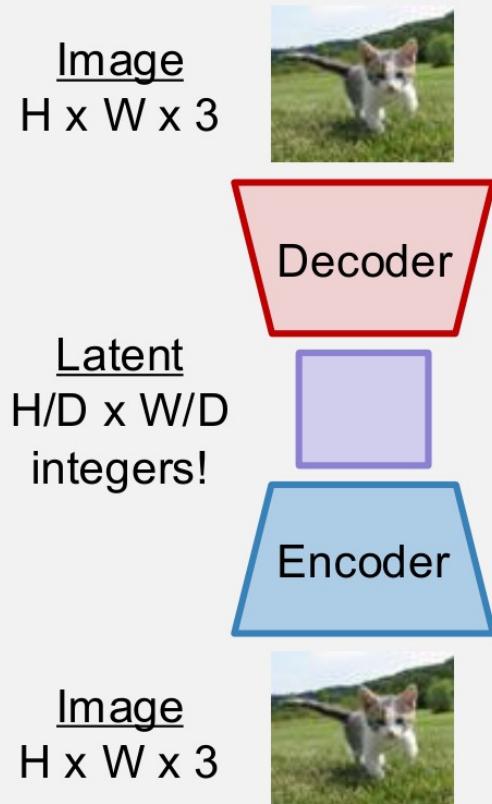
- Generate Multiple Tokens in Parallel → More ***Speed?!***
- Generate Tokens in any order → **Text editing?!**
- New probability paths → **Can we design ones that make semantic sense?**

Disadvantages

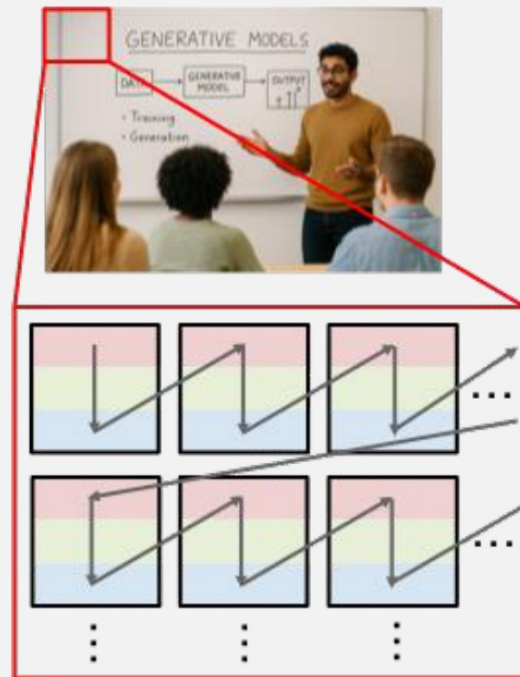
- No KV caching → **Less *Speed?!***
- Need to learn how to generate Tokens in *any* order → **Harder to learn?!**
- Autoregressive order (left-to-right) makes semantic sense → **Is it worth it?**

Autoregressive Models Strike Back

Train **encoder** + **decoder**
to convert images to
discrete latents



Train **autoregressive model**
to model sequences of
discrete latents



Sample discrete latents from
the **autoregressive model**,
pass to **decoder** to get an
image

van den Oord et al, "Neural Discrete Representation Learning", NeurIPS 2017
Razavi et al, "Generating Diverse High-Fidelity Images with VQ-VAE-2", NeurIPS 2019
Esser et al, "Taming Transformers for High-Resolution Image Synthesis", CVPR 2021
Yu et al, "Scaling Autoregressive Models for Content-Rich Text-to-Image Generation", arXiv 2022

Summary

- Flow matching = pick a path (straight line), regress the conditional velocity, sample by an ODE; diffusion is the Gaussian-path special case
- Straight paths and reflow make few steps work; flow maps / MeanFlow / drifting models make **one** step work
- 2026 generators: aligned VAE + MM-DiT + LLM text encoder + flow matching + CFG; the same recipe extends to video, audio and worlds
- RL fine-tuning needs stochastic sampling (Flow-GRPO); discrete flows give diffusion language models
- Reading: MIT 6.S184 lecture notes (2026); Lipman et al., 'Flow Matching Guide and Code' (2024)